Opisi algoritama

Zadatke, test primjere i rješenja pripremili: Tonko Sabolčec, Mislav Balunović, Ivan Paljak, Ivan Katanić i Ante Đerek. Primjeri implementiranih rješenja su dani u priloženim izvornim kodovima.

Zadatak: Paprike

Predložio: Ivan Paljak
Potrebno znanje: stablo, pohlepni algoritam, dinamičko programiranje

U terminima teorije grafova, cilj zadatka je obrisati što je moguće manji broj bridova ulaznog stabla tako da je u svakoj dobivenoj komponenti suma vrijednosti u čvorovima manja ili jednaka k.

Ukorijenimo stablo u proizvoljnom čvoru te sa $S(u)$ označimo sumu čvorova u podstablu čvora u (uključivo). Ako je $S(korijen) \leq k$, nije potrebno obrisati niti jedan brid. U protivnom, sigurno postoji čvor u pri čemu vrijedi $S(u) > k$ i $S(v) \leq k$ za svako dijete v čvora u. Očevidno je potrebno odrezati barem jedan brid iz podstabla čvora u.

Promatrajmo neko optimalno rješenje i pronađimo u njemu brid koji se nalazi u podstablu čvora u. Ako taj brid nije direktno povezan s u, dakle nalazi se u podstablu nekog djeteta v, umjesto njega mogli smo obrisati brid brid (u, v). Time dolazimo do zaključka da postoji optimalno rješenje u kom su svi bridovi podstabla čvora u incidentni s u.

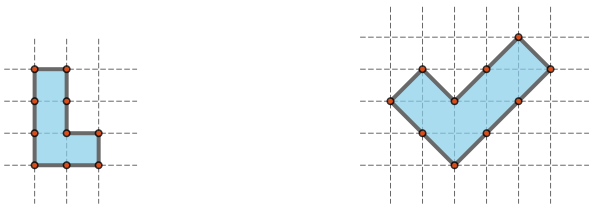
Obratimo sada pozornost na dijete w čvora u za koje je $S(w)$ maksimalan. Promatrajmo brid (u, v) koji se nalazi u optimalnom rješenju uz pretpostavku da se (u, w) ne nalazi u optimalnom rješenju. Zamijenimo li (u, v) sa (u, w) suma čvorova u komponenti čvora u će se smanjiti pa zaključujemo da ovu zamjenu možemo napraviti. Time dolazimo do zaključka da postoji optimalno rješenje u kojem se nalazi brid (u, w).

Rezanjem tog brida i primjenom istovjetnog algoritma na obje dobivene komponente dobivamo pohlepni algoritam implementiran u službenom rješenju. Razradu algoritma vremenske složenosti $O(n)$ ostavljamo čitateljici za vježbu.

Zadatak: Pick

Predložio: Ante Đerek
Potrebno znanje: geometrija, ad-hoc

Kao prvo, po teoremu opisanom u zadatku odmah lagano slijedi da, u ovom slučaju, poligon najmanje moguće površine je zapravo poligon koji sadrži najmanji mogući broj točaka s cjelobrojnim koordinatama u svojoj unutrašnjosti. Rješenje će, osim u jednom slučaju, uvijek konstruirati poligon koji ne sadrži niti jednu takvu točku u unutrašnjosti pa je stoga minimalne površine. Okomite i vodoravne segmente ćemo u ovom opisu rješenja zvati *ravni* segmenti, a dijagonalne *kosi* segmenti. Točke s cjelobrojnim koordinatama ćemo jednostavno zvati *točke*.



Slika 1: Oblici rješenja u prvom i drugom podzadatku.

Razmatrajmo prva dva podzadatka. Ako je $c = d = 0$ (dakle na raspolaganju imamo samo ravne segmente) onda broj jednih i drugih mora biti paran i u tom slučaju možemo konstruirati poligon u obliku slova "L"

koji nema niti jednu točku unutra pa je stoga minimalan. Ako je $c = d = 0$ onda je situacija analogna, rješenje je rotirano slovo "L". U ovom slučaju se ne postiže da unutra nema cjelobrojnih točaka, i to je ujedno i jedini slučaj kada ne možemo konstruirati poligon koji nema niti jednu točku unutra.

Sada pretpostavljamo da nam je na raspolaganju barem jedan kosi i barem jedan ravni segment te razmatramo koji su mogući ostaci pri dijeljenu s dva brojeva a, b, c i d. Segmenti tipa a, c i d mijenjaju parnost x koordinate pa zbroj $a + c + d$ uvijek mora biti paran. Sličnim načinom razmišljanja lagano se vidi da postoje četiri mogućnosti za niz ($a \bmod 2, b \bmod 2, c \bmod 2, d \bmod 2$) i to su redom:

1. $(0, 0, 0, 0)$
2. $(0, 0, 1, 1)$
3. $(1, 1, 0, 1)$
4. $(1, 1, 1, 0)$

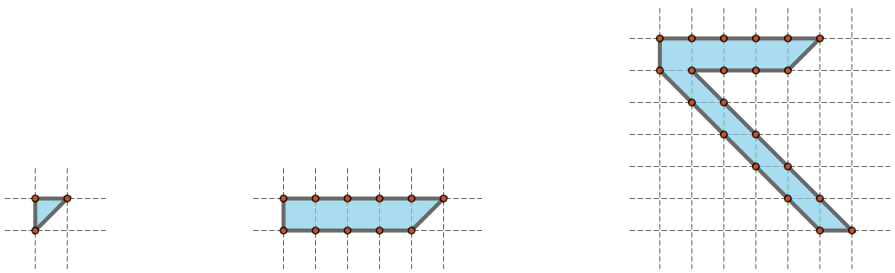
Konstrukcija sada teče u dva koraka: u prvom koraku napravimo takozvani *početni* poligon — neki minimalni poligon koristeći 3 ili 4 segmenta tako da nam preostane *paran* broj segmenata svakog tipa. U drugom koraku poligon proširujemo dodajući "tornjeve" od preostalih segmenata.

U slučaju 1. gore, postoje barem dva ravna segmenta istog tipa i barem dva kosa segmenta istog tipa pa pomoću njih konstruiramo paralelogram površine 1. U slučaju 2. postoje barem dva ravna segmenta istog tipa pa pomoću njih i dva različita kosa segmenta konstruiramo trokut minimalne površine. U slučajevima 3. i 4. od po jednog "neparnog" segmenta konstruiramo trokut minimalne površine. Primjer sva četiri slučaja dan je na slici 2.



Slika 2: Konstrukcija početnog poligona.

U drugom koraku konstrukcije, na raspolaganju imamo početni poligon te paran broj segmenata svakog tipa koje je potrebno "potrošiti". Razmatramo jedan po jedan tip i od parnog broja segmenata radimo "tunel" koji dodajemo postojećem poligonu. Tunel dodajemo tako da odaberemo jedan postojeći brid, uklonimo ga, na njegovo mjesto dodamo dva jednako dugačka segmenta te zatvorimo tunel pomoću uklonjenog brida. Na slici 3 dodajemo 8 segmenata tipa a pa zatim 10 segmenata tipa d na opisani način.



Slika 3: Dodavanje tunela.

Primijetite da ovaj korak ne mora uvijek uspjeti. Moguće je dobiti poligon koji nije minimalan (ako su i tunel i uklonjeni brid kosi), ili crtu koja uopće nije poligon jer recimo siječe samoga sebe. Međutim, razmatrajući slučajeve moguće je se uvjeriti da je stvarno uvijek moguće na ovaj način "potrošiti" bridove.

Prilikom implementacije rješenja, umjesto razmatranja slučajeva, možemo probati na sve moguće načine dodati tunel dok ne dobijemo ispravan poligon. Jedno pazimo da ne uklanjamo kosi brid ako se radi o kosom tunelu kako bi uvijek dobili minimalan poligon. Na ovaj način radi priloženo rješenje.

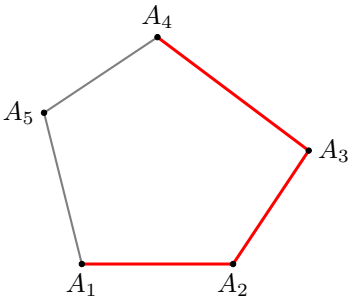
Općenito, postoji puno različitih pristupa rješavanju ovog zadatka. Svako efikasno rješenje zaključi neka svojstva minimalnog poligona, ali i onda u nekim koracima koristi isprobavanje svih kombinacija što je lakše implementirati nego mukotrpno razmatrati moguće slučajeve. Jedan trik koji uvelike olakšava implementaciju je da poligon ne pamtimo kao niz točaka nego kao niz tipova bridova tako da, na primjer, znak "`a`" označava brid tipa a koji ide slijeva nadesno, a znak "`A`" brid koji ide u suprotnom smjeru.

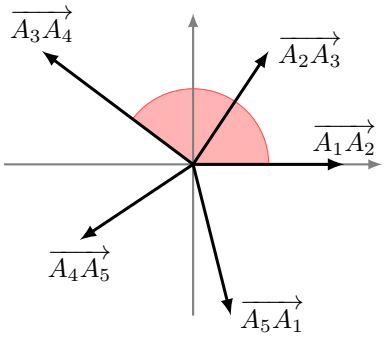
Opišimo za kraj i jednostavno rješenje koje može osvojiti djelomične bodove. U prvom koraku konstruiramo veliki poligon koristeći zadane bridove tako da jednostavno idemo "u krug". U ovom koraku možemo koristiti isprobavanje svih kombinacija (koliko bridova tipa a ide lijevo odnosno desno) kako bi jednostavno pronašli bilo koji poligon. U drugom koraku pokušavamo iterativno poboljšati rješenje koristeći jednostavne slučajne transformacije (primjerice zamjenu neka dva brida). Postupak završavamo netom prije nego što naše rješenje pređe preko vremenskog ograničenja te ispišemo najbolje pronađeno rješenje. Ovim pristupom je bilo moguće osvojiti oko 60% bodova.

Zadatak: Svjetlost

Predložio: Tonko Sabolčec
Potrebno znanje: geometrija, cirkularni sweep-line, strukture podataka

Radi lakseg shvacanja rjesenja, promatrajmo duzine kao vektore kao sto je prikazano na slikama ispod.





Primijetimo da će izvor svijetlosti uvijek osvjetljavati uzastopni niz dužina. Također, bez obzira na odabir točke T, kut koji će zatvarati početna i završna osvijetljena dužina uvijek će biti strogo manji od 180 stupnjeva. Zadatak se zapravo svodi na to da za svaki uzastopni niz dužina koje zatvaraju kut manji od 180 stupnjeva uzmemo onaj čija je suma duljine dužina najveća.

Rješenje koje za svaki poligon fiksira početnu dužinu i u niz dodaje dužine dok god je kut koji zatvara s početnom dužinom manji od 180 stupnjeva ima slozenost $O(qn^2)$ što je dovoljno da se riješi prvi podzadatak. Provjeru ako neke dvije dužine zatvaraju kut manji, veći ili jednak 180 stupnjeva možemo postići korištenjem *ccw* (counter-clockwise) funkcije koja je definirana na sljedeći način:

$$ccw(A, B, C) = x_A(y_B - y_C) + x_B(y_C - y_A) + x_C(y_A - y_B),$$

gdje su A, B i C neke točke. Vrijednost funkcije pozitivna je ako su točke A, B i C poredane u smjeru suprotnom od smjera kazaljke na satu, negativna ako su poredane u smjeru kazaljke na satu ili je jednaka nuli ako su kolinearne. Ako dužine $\overline{A_iA_{i+1}}$ i $\overline{A_jA_{j+1}}$ zatvaraju kut manji od 180 stupnjeva, tada vrijedi $ccw((0,0), \overrightarrow{A_iA_{i+1}}, \overrightarrow{A_jA_{j+1}}) > 0$.

Odgovore za pojedini poligon možemo ubrzati pomoću cirkularnog sweep-line algoritma. Recimo da smo odredili početni interval dužina koje zatvaraju kut manji od 180 stupnjeva, od točke A_l do A_r. U sljedećih n iteracija ponavljamo sljedeće korake:

1. Smanji inteval dužina micanjem točke A_l, tj. povećaj l za 1.
2. Povećavaj r za 1 dokle god će interval dužina zatvarati kut manji od 180 stupnjeva.

Nakon točno n iteracija doći ćemo do intervala od kojeg smo i krenuli, tj. naći ćemo optimalno osvijetljenje za svaku početnu točku. Obje varijable l i r smo povećali točno n puta, pa je ukupna složenost ovog algoritma $O(qn)$.

Neka je $izvor(k)$ skup dužina počevši od točke A_k koje će osvjetljavati optimalno odabran izvor svjetlosti. Primjerice, osvijetljen dio dužina na prvoj slici odgovara skupu $izvor(0)$. Primijetimo da će neku dužinu osvijetljavati uzastopni niz izvora.

Zadatak možemo riješiti pomoću tournament strukture podataka u kojem kranjnji čvorovi predstavljaju vrijednosti izvora - tj. sumu duljina svih dužina u tom izvoru. Brisanjem točke k iz trenutnog poligona događaju se sljedeće operacije:

- Izbaci dužinu $A_kA_{p_k}$,
- izbaci dužinu $A_kA_{n_k}$ i
- dodaj dužinu $A_{p_k}A_{n_k}$.

Točke A_{p_k} i A_{n_k} su susjedne odabranoj točki A_k u poligonu. Svaka operacija zahvatit će određeni uzastopni niz izvora. Prve dvije operacije smanjuju vrijednost tih izvora, dok treća operacija povećava. Drugim riječima, svaka od tih operacija svodi se na dodavanje neke vrijednosti svim čvorovima tournamenta u intervalu izvora koji osvjetljavaju dužinu.

Do odgovora nakon svakog brisanja možemo doći tako da među svim izvorima, uzmemo onaj s najvećom vrijednošću. Drugim riječima, zanima nas maksimalna vrijednost nekog čvora u tournamentu na cijelom intervalu. Tournament koji podržava navedene upite moguće je ostvariti tehnikom lijenog propagrianja.

Interval izvora koji osvjetaljaju neku dužinu može se naći pomoću binarnog pretraživanja ili modifikacijom na prije opisanom algoritmu cirkularnog sweep-line-a. Ukupna složenost algoritma je $O((n+q)\log n)$.

Zadatak: Zagonetka

Predložio: Mislav Balunović
Potrebno znanje: teorija grafova, ad-hoc

Konstruirajmo sljedeći graf sa čvorovima od 1 do n. Dodajmo *usmjereni* brid između čvorova a i b ako vrijedi $p_a < p_b$ za svaku permutaciju p za koju smo dosada dobili odgovor (uključujući i početnu permutaciju). Kažemo da brid (a, b) *postoji* ako je $(a, b) \in S$.

Svaki brid koji može biti jedan od sljedeća dva tipa:

- tip * koji znači da brid sigurno postoji
- tip ? koji znači da nismo sigurni postoji li brid ili ne

Također, ako imamo bridove (a, b) i (b, c) koji su tipa * onda ćemo pretpostaviti i da je brid (a, c) tipa *. Naime, ako su (a, b) i (b, c) oba u skupu S, onda nam je svejedno nalazi li se (a, c) u S jer to ne mijenja skup P koji S proizvodi. Zato ćemo pretpostaviti da je $(a, c) \in S$.

Na početku su svi bridovi tipa ?. Sve dok postoji barem jedan brid tipa ? radimo sljedeće:

1. Želimo pronaći neki brid (a, b) tipa ? takav da ne postoji neki niz $x_1, ..., x_k$ takav da postoje bridovi $(a, x_1), (x_1, x_2), ..., (x_k, b)$. Jedan način za pronaći takav par je napraviti *topološki sort* grafa te uzeti prvi brid tipa ? koji se pojavi u topološkom poretku.
2. Kada imamo takav brid (a, b) možemo ga privremeno obrisati iz grafa, umjesto njega dodati brid (b, a) te pronaći bilo koju permutaciju q koja zadovoljava sve bridove, npr. topološkim sortom. Primijetite da takva permutacija postoji jer okretanjem brida (a, b) sigurno nismo napravili ciklus u grafu zbog načina na koji je (a, b) odabran.

3. Postavimo upit sa permutacijom q. Ako dobijemo odgovor 1 znači da nam je brid (a, b) nepotreban pa ga obrišemo iz grafa. Ako dobijemo odgovor 0 znači da brid sigurno postoji pa pretvaramo njegov tip iz ? u *.

Primijetite da će u svakom koraku ili jedan brid biti obrisan ili će tip jednog brida biti pretvoren iz ? u * pa to znači da ćemo napraviti najviše $O(\frac{n\cdot(n-1)}{2})$ upita.

Konačno, kada za svaki brid znamo postoji li ili ne, preostaje izračunati leksikografski najmanju i najveću permutaciju koja zadovoljava uvjete. Jedan mogući način za pronaći leksikografski najmanju permutaciju je pomoću rekurzivne funkcije $gen(V, a, b)$ koja računa najmanju permutaciju koju možemo pronaći za neki podskup čvorova V ako svi oni dobivaju brojeve iz intervala $[a, b]$.

Funkcija radi tako da uzmemo najmanji $x \in V$ te označimo sa A skup svih čvorova iz kojih postoji brid u x. Tada poziciji x dodijelimo vrijednost $k = a + |A|$ (koja je očito najmanja moguća). Onda rekurzivno pozovemo $gen(A, a, k-1)$ i $gen(V \setminus (A \cup \{x\}), k, b)$. Permutacija koju tražimo dobije se pozivom $gen(\{1, 2, ..., N\}, 1, N)$. Najveća permutacija generira se analogno.